# Software Engineering I (02161)

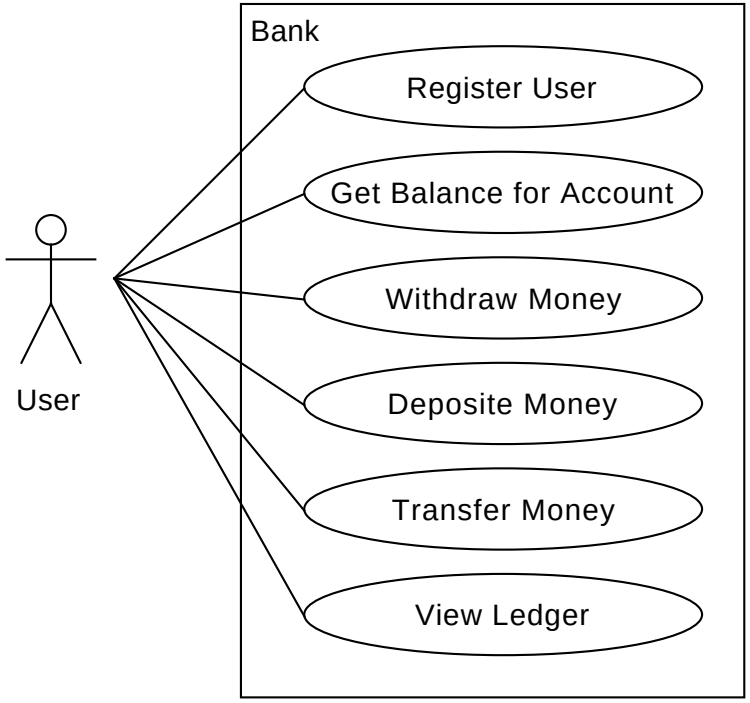
## First Programming Assignment

Assoc. Prof. Hubert Baumeister

DTU Compute
Technical University of Denmark

# Programming Exercise: Bank developed TDD

- Develop a Bank class with methods register, getBalance, withdraw, deposit, transferMoney, and getTransactions (advanced)



# Test-Driven-Development TDD

- Sometimes also referred to as Behaviour-Driven Development (BDD)
- First write a test for the correct behaviour, then write the production code.
  - *Given that a user is registered with a non zero balance. When the user then withdraws money from the bank using his bank account number, then the balance is reduced correspondingly..*

```
Bank bank = new Bank();
String bankNumber = bank.register( userName: "John Doe",  initialBalance: 1000);
bank.withdraw(bankNumber,  amount: 200);
assertEquals( expected: 800,bank.getBalance(bankNumber));
```

# JUnit test vs Cucumber test

- JUnit: Uses Java; not readable by customer

```java
public class TestBank {  new *
    @Test  new *
    public void testWithdraw() throws Exception {
        Bank bank = new Bank();
        String bankNumber = bank.register( userName: "John Doe",  initialBalance: 1000);
        bank.withdraw(bankNumber,  amount: 200);
        assertEquals( expected: 800,bank.getBalance(bankNumber));
    }
}
```

- Cucumber: Uses Gherkin; readable by customer

```gherkin
Feature: Managing Withdrawals in Bank Application
  As a bank user,
  I want to withdraw money from my account,
  So that my account balance is reduced accordingly.

  Scenario: User withdraws money successfully
    Given a registered user with an account balance of 1000
    When the user withdraws 200
    Then the account balance should be 800
```

# Step Definitions

```
public class BankSteps {  new *
    Bank bank = new Bank();  3 usages
    String bankNumber;  3 usages

    @Given("a registered user with an account balance of {int}")  new *
    public void aRegisteredUserWithAnAccountBalanceOf(Integer balance) {
        bankNumber = bank.register( userName: "John Doe", balance);
    }

    @When("the user withdraws {int}")  new *
    public void whenTheUserWithdraws(Integer amount) throws Exception {
        bank.withdraw(bankNumber, amount);
    }

    @Then("the account balance should be {int}")  new *
    public void thenTheAccountBalanceShouldBe(Integer expectedBalance) throws Exception {
        assertEquals(expectedBalance, bank.getBalance(bankNumber));
    }
}
```

# Test-Driven Development: red-green-refactor

- write **one** failing test (red)
- write the production code that makes the test pass (green)
- make the code ”beautiful” (refactor)

- **Important:** Only write the code that makes the test pass
  - If you want to write more code, first make a corresponding test